

Experiment ‘p_4_2_b_count_even_odd_array’ Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_4_2_b_count_even_odd_array

Natural language statement: null

Method signature: p_4_2_b_count_even_odd_array(inputs: array<int>) returns (even_count: int, odd_count: int)

Ensures

- even_count == number_even(inputs[..])
- odd_count == number_odd(inputs[..])

Functional Code Given

```
function number_odd(s: seq<int>): int
{ if s == [] then 0
  else if s[0] % 2 == 1 then 1 + number_odd(s[1..])
  else number_odd(s[1..])
}

function number_even (s: seq<int>): int
{ if s == [] then 0
  else if s[0] % 2 == 0 then 1 + number_even(s[1..])
  else number_even(s[1..])
}
```

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_4_2_b_count_even_odd_array(inputs: array<int>) returns (even_count: int, odd_count: int`
→ `)`

The method should respect the following contract:

`ensures even_count == number_even(inputs[..]), ensures odd_count == number_odd(inputs`
→ `[..])`

The contract uses the following dafny code:

```
function number_odd(s: seq<int>): int
{ if s == [] then 0
  else if s[0] % 2 == 1 then 1 + number_odd(s[1..])
  else number_odd(s[1..])
}
```

```
function number_even (s: seq<int>): int
{ if s == [] then 0
  else if s[0] % 2 == 0 then 1 + number_even(s[1..])
  else number_even(s[1..])
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnLJIg01p1kT023uaC0vXOJCv6Wiq

Model: gpt-5.5-2026-04-23

Created at: 1780652360

Finish reason: stop

Usage: 558 tokens in, and 3619 tokens out

```
{
  even_count := 0;
  odd_count := 0;
```

```

var i := inputs.Length;

while i > 0
  invariant 0 <= i <= inputs.Length
  invariant even_count == number_even(inputs[i..])
  invariant odd_count == number_odd(inputs[i..])
  decreases i
{
  i := i - 1;
  assert 0 <= i < inputs.Length;
  assert even_count == number_even(inputs[i + 1..]);
  assert odd_count == number_odd(inputs[i + 1..]);
  assert inputs[i..][0] == inputs[i];
  assert inputs[i..][1..] == inputs[i + 1..];

  if inputs[i] % 2 == 0 {
    assert number_even(inputs[i..]) == 1 + number_even(inputs[i + 1..]);
    assert number_odd(inputs[i..]) == number_odd(inputs[i + 1..]);
    even_count := even_count + 1;
  } else {
    assert 0 <= inputs[i] % 2 < 2;
    assert inputs[i] % 2 == 1;
    assert number_even(inputs[i..]) == number_even(inputs[i + 1..]);
    assert number_odd(inputs[i..]) == 1 + number_odd(inputs[i + 1..]);
    odd_count := odd_count + 1;
  }
}

assert i == 0;
assert inputs[i..] == inputs[..];
}

```

Final Program

```

function number_odd(s: seq<int>): int
{ if s == [] then 0
  else if s[0] % 2 == 1 then 1 + number_odd(s[1..])
  else number_odd(s[1..])
}

function number_even (s: seq<int>): int
{ if s == [] then 0
  else if s[0] % 2 == 0 then 1 + number_even(s[1..])
  else number_even(s[1..])
}

method p_4_2_b_count_even_odd_array(inputs: array<int>) returns (even_count: int,
  ↪ odd_count: int)
  ensures even_count == number_even(inputs[..])
  ensures odd_count == number_odd(inputs[..])

```

```

{
  even_count := 0;
  odd_count := 0;
  var i := inputs.Length;

  while i > 0
    invariant 0 <= i <= inputs.Length
    invariant even_count == number_even(inputs[i..])
    invariant odd_count == number_odd(inputs[i..])
    decreases i
  {
    i := i - 1;
    assert 0 <= i < inputs.Length;
    assert even_count == number_even(inputs[i + 1..]);
    assert odd_count == number_odd(inputs[i + 1..]);
    assert inputs[i..][0] == inputs[i];
    assert inputs[i..][1..] == inputs[i + 1..];

    if inputs[i] % 2 == 0 {
      assert number_even(inputs[i..]) == 1 + number_even(inputs[i + 1..]);
      assert number_odd(inputs[i..]) == number_odd(inputs[i + 1..]);
      even_count := even_count + 1;
    } else {
      assert 0 <= inputs[i] % 2 < 2;
      assert inputs[i] % 2 == 1;
      assert number_even(inputs[i..]) == number_even(inputs[i + 1..]);
      assert number_odd(inputs[i..]) == 1 + number_odd(inputs[i + 1..]);
      odd_count := odd_count + 1;
    }
  }

  assert i == 0;
  assert inputs[i..] == inputs[..];
}

```

Total Token Usage

Input tokens: 558

Output tokens: 3619

Reasoning tokens: 3214

Sum of 'total tokens': 4177

Experiment Timings

Overall Experiment started at 1780652360235, ended at 1780652452562, lasting 92327ms (92.33 seconds)

Iteration #1 started at 1780652360240, ended at 1780652452562, lasting 92322ms (92.32 seconds)